

Project Requirements Specification (GROUP K)

Project Title: Smart Voice-Activated Campus Guide Kiosk

Project Type: Human-Machine Interface (HMI) & Progressive Web Application (PWA)

1. Project Overview

This project aims to replace a traditional mobile guide robot with a static, interactive kiosk located at the school entrance. The system utilizes voice recognition to understand visitor inquiries about campus locations and generates a dynamic QR code. When scanned, this QR code launches a custom map application on the visitor's smartphone, providing turn-by-turn navigation to their specific destination.

2. Hardware Requirements (The Kiosk Console)

The physical unit that users interact with.

Component	Specification	Purpose	Cost()
Microcontroller	Raspberry Pi 4 Model B (4GB or 8GB RAM)	The brain of the kiosk. 4GB+ RAM is required to run the React application and Chromium browser smoothly in Kiosk mode.	90,000
Display	Raspberry Pi Official 7-inch Touchscreen (or 10" HDMI Capacitive Touch Display)	Provides the visual interface for the user and displays the QR codes.	80,000
Audio Input	USB Conference Microphone (Omnidirectional)	Captures user speech from a distance (1-2 meters) without needing to hold a device.	7,000
Audio Output	3.5mm Aux Speakers or USB Powered Speakers	Provides voice feedback (Text-to-Speech) to the user (e.g., "Scanning route...").	7,000
Power Supply	Official Raspberry Pi USB-C Power Supply (15.3W)	Ensures stable voltage to prevent system crashes during processing.	10,000
Storage	32GB Class 10 MicroSD Card	Stores the Operating System and local application files.	20,000

Component	Specification	Purpose	Cost()
Raspberry Pi heat sink	Cooling solution with fans	For cooling the Raspberry Pi	10,000
Printer	TTL Serial Thermal Printer (Xprinter XP-58IIH)	To handle users that do not have a mobile device	50,000
Casing	Made out of wood	Holds the screen at a 45-degree viewing angle for ergonomic interaction.	

3. Software Requirements

A. The Console Application (Kiosk Interface)

Running on the Raspberry Pi.

- **Operating System:** Raspberry Pi OS (64-bit Desktop Version).
- **Browser Environment:** Chromium Browser configured in `--kiosk` mode (fullscreen, no address bar, disable gestures).
- **Framework:** **React.js** (Vite build tool recommended for performance).
- **Speech Recognition:** **Web Speech API** (`window.SpeechRecognition`) for converting voice to text.
- **Speech Synthesis:** **Web Speech API** (`window.speechSynthesis`) for voice response.
- **QR Generation:** `qrcode.react` library to dynamically generate Deep Links.
- **Network:** Wi-Fi connection (Required for Web Speech API accuracy).

B. The Mobile Map Application (User's Phone)

Running on the visitor's smartphone after scanning.

- **Type:** **Progressive Web App (PWA)**.
- **Framework:** React.js.
- **Mapping Library:** **Leaflet.js** (via `react-leaflet`).
- **Map Tiles Provider:** **OpenStreetMap (OSM)** (Free, open-source).
- **Routing Engine:** **Leaflet Routing Machine** (plugin) to draw paths from the Gate to the destination.
- **Deep Linking:** Logic to parse URL parameters (e.g., `?destination=admin_block`) to auto-start navigation.

4. Data Acquisition Strategy (Geolocation)

To ensure the map is accurate, coordinates for all campus buildings must be captured manually.

Selected Tool: **SW Maps (Android)**

Alternative: Epicollect5 or Google Maps (Satellite View)

Workflow:

1. **Surveying:** Walk to the entrance of every key building (Hostels, Admin, Library, Cafeteria).
2. **Recording:** Use **SW Maps** to record a "Point Feature" at the entrance.
3. **Labeling:** Name the point clearly (e.g., "Library_Main_Entrance").
4. **Export:** Export the data as a **GeoJSON** or **CSV** file.
5. **Integration:** Import this file directly into the React Map Application database
(`locations.json`).

5. Functional Requirements

5.1 Idle Mode

- The system shall display a welcoming interface with a pulsing microphone icon.
- The system shall display text prompts (e.g., "Tap here or say 'Hello' to start").

5.2 Interaction Mode

- The system shall activate the microphone upon detecting a "Wake Word" or a screen tap.
- The system shall visualize voice input (waveforms) to indicate it is listening.
- The system shall parse the spoken sentence for keywords matching the internal location database.

5.3 Error Handling

- If the location is not found, the system shall respond verbally: "I'm sorry, I didn't catch that. Please try 'Library' or 'Cafeteria'."
- If the internet is disconnected, the system shall display a "Offline - Manual Mode" allowing users to select destinations via touch buttons.

5.4 Output Mode

- The system shall generate a specific URL containing the destination ID (e.g., `https://schoolmap.app/?to=library`).
- The system shall render this URL as a high-contrast QR code.
- The system shall provide a verbal confirmation: "Here is the map to the Library. Please scan the code."

5.5 Mobile Navigation

- Upon scanning, the Mobile App shall open instantly.
- The app shall detect the `?to=` parameter.
- The app shall calculate and display a walking path from the "Main Gate" (Fixed Point) to the requested location.

6. Non-Functional Requirements

- **Latency:** Voice recognition to QR generation should take less than 3 seconds.
- **Accessibility:** The interface should use large fonts and high contrast for visibility.
- **Reliability:** The Mobile Map PWA must utilize Service Workers to cache map tiles, ensuring it works on visitor phones even with poor internet connection on campus.
- **Ergonomics:** The screen should be positioned at a height accessible to average-height adults.